Both prompt compression and k-means clustering offer unique advantages for document summarization, but a balanced approach is recommended. A hybrid strategy, combining both, can enhance performance and robustness while testing and accepting feedback to help address any shortcomings and improve outcomes. The code examples should give you a good overview of how to implement these summarization techniques in your own projects.

7.2 Building a RAG web application with LangChain.js

Let's move on to our second project, which makes use of retrieval-augmented generation (RAG) techniques. RAG is an advanced natural language processing technique that enhances LLMs by retrieving relevant information from external knowledge bases before generating a response. In simple terms, it injects real-time information retrieval into the workflow so the model can answer queries with fresh, topic-specific context. By supplying the LLM with this retrieved context for each query, RAG significantly increases the accuracy and relevance of its responses. Instead of relying solely on pretrained, potentially outdated knowledge, the model can incorporate the latest data, facts, and insights that are directly related to the user's inquiry.

7.2.1 RAG app project requirements

This app uses RAG to enhance the accuracy and relevance of generated responses by integrating real-time information retrieval with generative capabilities.

Here's what we'll be building:

- *User interface*—We will create a user-friendly interface using Next.js and the Vercel AI SDK to facilitate user interactions.
- *Document processing*—We will implement the LangChain.js library to process and manage documents, ensuring they are properly sanitized and chunked for efficient retrieval.
- *Query processing*—Using LangChain.js, we will develop a robust query-processing mechanism that includes input handling, document retrieval, and ranking of relevant information.
- *Augmentation layer*—We will integrate the retrieved context with user queries through LangChain.js, preparing an enriched prompt for the generation engine.
- *Response generation*—Finally, we will utilize the capabilities of LLMs to generate coherent and contextually relevant responses based on the augmented prompts. We will be presenting this into the familiar UI interface for the user to interact with.

Once we implement these steps, we will create a comprehensive RAG system that effectively combines document retrieval and generation capabilities. Before we look at the technical requirements of the project, let's learn a bit more about RAG and its key architectural components.